

Extracurriculars | EC

Summary

CONTENTS

- 1. Category Theory in Context - Emily Rie 2
 - 1.1. Chapter 1 2
 - 1.1.1. Abstract and concrete categories 2
 - 1.1.2. Duality 4
- 2. OSDEV 6
 - 2.1. Kernel 6
 - 2.1.1. Memory 6
 - 2.1.2. Processes 7
 - 2.1.3. Syscalls 7
 - 2.1.4. CPU 7
 - 2.1.5. Filesystem 8
 - 2.1.6. Hardware 8

1. CATEGORY THEORY IN CONTEXT - EMILY RIE

Notation:

- Morphism
- $\Rightarrow$ Parallel pair of morphisms
- $\rightrightarrows$ Opposing pair of morphisms
- $\mapsto$ "maps to"
- $\rightsquigarrow$ "yields" / "leads to"
- $\leadsto$ Implied existence of another morphism
- $=$ Genuine equality
- $:=$ Definitional equality
- $\cong$ Isomorphism
- $\simeq$ Weaker equivalence
- $\rightarrow$ Epimorphism
- $\hookrightarrow$ Inclusion/Embedding/"beinhaltet"/ $\subset$?

1.1. CHAPTER 1

1.1.1. Abstract and concrete categories

Term	Definition
Commutative	The order of the elements in an operation does not affect the result
Commutative diagram	Diagram where all directed paths with the same starting and ending points yield the same result
Natural	
Functor	

Groups

Term	Definition
Group	A non-empty set with a binary operation satisfying associativity, identity and inverse $\Leftrightarrow$ A groupoid with one object
Subgroup	
Group extension	
Abelian group	A Group equipped with an operation that satisfies the commutative property
Abelian extension	
Ring	
Quiver	A directed graph defined by objects and morphisms in a category

Morphisms

Term	Definition
Morphism	A structure-preserving map between two objects within a category
Homomorphism	A morphism mapping between two algebraic structures while preserving their operations

<i>Term</i>	<i>Definition</i>
Isomorphism	A morphism $f : X \rightarrow Y$ for which there exists a morphism $g : Y \rightarrow X$ so that $gf = 1_X \wedge fg = 1_Y$ A morphism which is both a monomorphism and an epimorphism. *
Endomorphism	A morphism whose domain equals its codomain
Automorphism	An isomorphic endomorphism

Categories

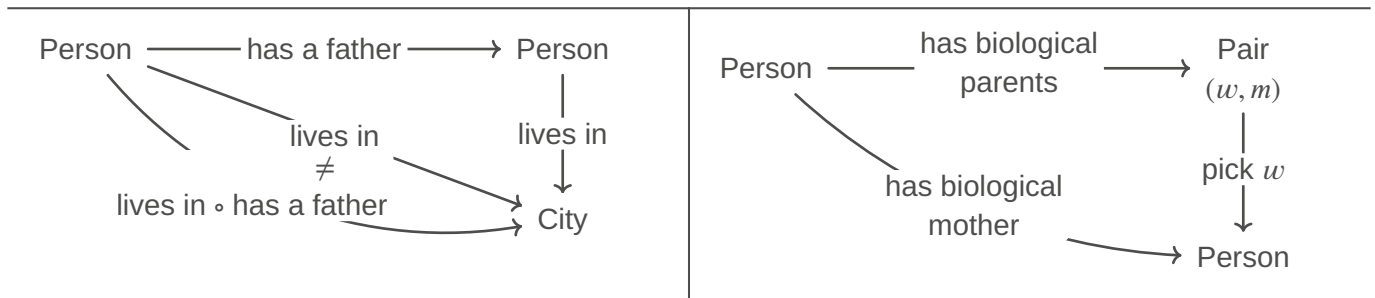
<i>Term</i>	<i>Definition</i>
Category	Consists of objects $X, Y, Z, \dots$ and morphisms $f, g, h, \dots$, such that: – Each morphism has a specified domain and codomain among the collection of objects – Each object has a designated identity morphism – For any composable pair of morphisms g, f there exists a composite morphism gf whose domain is equal to the domain of f and whose codomain is equal to the codomain of g
Subcategory	Subcollection of objects and morphisms of a category, such that it contains the domain and codomain of any morphism, the identity morphism of any object and the composite of any composable pair of morphisms in the subcategory
Small category	A category is small if it has only a set's worth of arrows
Locally small category	A category is locally small if between any pair of objects there is only a set's worth of morphisms
Hom-set	Set of arrows between a pair of fixed objects in a locally small category
Discrete Category	Every morphism is an identity
Slice category	

Groupoids

<i>Term</i>	<i>Definition</i>
Groupoid	A category in which every morphism is an isomorphism $\Rightarrow$ has itself as its opposite category. Example: Discrete Category
Fundamental groupoid	For any space X , its fundamental groupoid $\Pi_1 X$ is a category whose objects are the points of X and whose morphisms are endpoint-preserving homotopy classes of paths
Maximal groupoid	The subcategory containing all of the objects and only those morphisms that are isomorphisms
Comonoid	

1.1.1.1. Book club (04.03.26)

*Not commutative**Commutative*



1.1.1.1.1. Let $f : x \rightarrow y$. Show that if $g, h : y \rightrightarrows x$ so that $gf = 1_x$ and $fh = 1_y$ then $g = h$ and f is an isomorphism.

(i) $a) f \circ h = 1_y$
 $b) g \circ f = 1_x$
 $\quad \quad \quad \downarrow$
 $\quad \quad \quad g = h$

$$\begin{aligned} g &= g \circ 1_y && | a) \\ &= g \circ (f \circ h) \\ &= (g \circ f) \circ h && | b) \\ &= 1_x \circ h \\ &= h \end{aligned}$$

(ii)

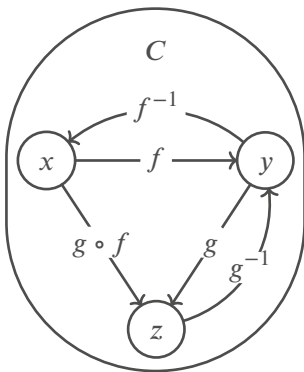
1.1.1.1.2. Let C be a category. Show that the collection of isomorphisms in C defines a subcategory, the maximal groupoid inside C

There should exist a Category D , such that:

- Elements in D : The same as in C
- Morphisms in D : The isomorphisms in C

All identity morphisms can be taken into D

Given isomorphisms $f : x \rightarrow y, g : y \rightarrow z$, show that $g \circ f$ is isomorph.



$$\begin{aligned} (g \circ f)^{-1} &= f^{-1} \circ g^{-1} \\ \Leftrightarrow (g \circ f)^{-1} \circ (g \circ f) &= 1_x \\ \Leftrightarrow (g^{-1} \circ f^{-1}) \circ (g \circ f) &= 1_x \\ \Leftrightarrow g^{-1} \circ (f^{-1} \circ g) \circ f &= 1_x \\ \Leftrightarrow g^{-1} \circ 1_y \circ f &= 1_x \\ \Leftrightarrow g^{-1} \circ f &= 1_x \\ \Leftrightarrow 1_x &= 1_x \end{aligned}$$

$$\begin{aligned} (g \circ f) \circ (g \circ f)^{-1} &= 1_z \\ \Leftrightarrow g \circ f \circ f^{-1} \circ g^{-1} &= 1_z \\ \Leftrightarrow g \circ 1_y \circ g^{-1} &= 1_z \\ \Leftrightarrow g \circ g^{-1} &= 1_z \\ \Leftrightarrow 1_z &= 1_z \end{aligned}$$

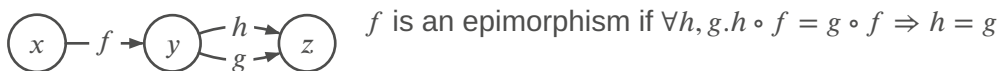
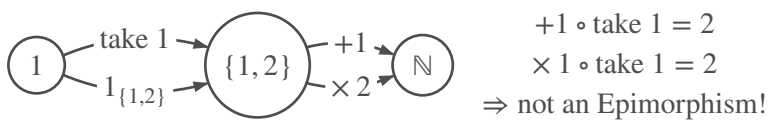
1.1.2. Duality

Term	Definition
Duality	
Opposite Category	An opposite category of C , called C^{op} , has the same objects as C and a morphism f^{op} for each morphism f in C so that the domain of f^{op} is defined to be the codomain of f and the codomain of f^{op} is defined to be the domain of f
Composable	
Postcomposition	$f_* : C(c, x) \rightarrow C(c, y), c \in C$
Precomposition	$f^* : \underbrace{C(y, c)}_{\substack{\text{Set of morphisms /} \\ \text{"higher order morphisms"}}} \rightarrow C(x, c), c \in C$

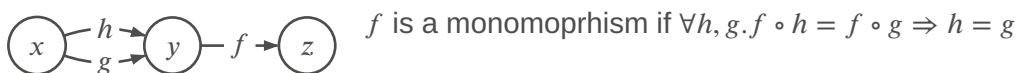
Term	Definition
Monomorphism	
Epimorphism	
Supremum	
Infimum	
section/right inverse	
retraction/left inverse	
retract	
split morphism	

1.1.2.1. Book club (11.03.26)

1.1.2.1.1. Ex 1

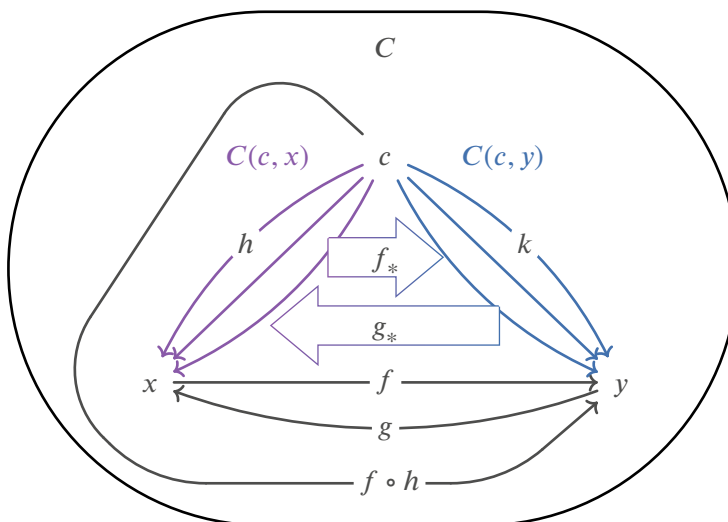


1.1.2.1.2. Ex 2



1.1.2.2. Book club (18.03.26)

1.1.2.2.1. Lemma 1.2.3

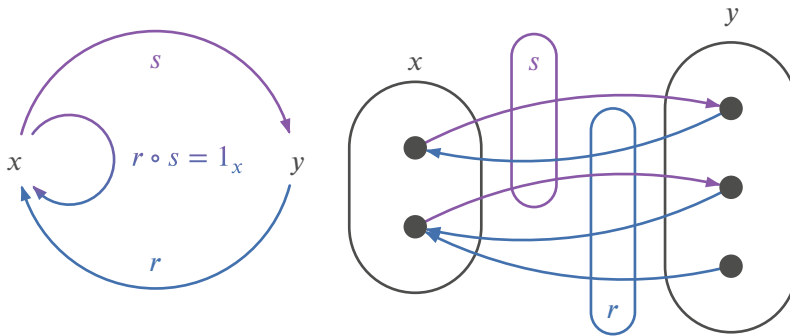


$$\begin{aligned}
 (i) &\stackrel{!}{\Rightarrow} (ii) \\
 (1) &\begin{cases} f_*(h) := f \circ h \\ f_* : h \mapsto f \circ h \end{cases} \\
 (2) &\begin{cases} g_*(k) := g \circ k \\ g_* : k \mapsto g \circ k \end{cases} \\
 &\left. \begin{aligned} g_*(f_*(h)) &\stackrel{(1)}{=} g_*(g \circ h) \stackrel{(2)}{=} \underbrace{g \circ f \circ h}_{(i) \Rightarrow 1_x} \stackrel{(i)}{=} h \\ f_*(g_*(k)) &\stackrel{(2)}{=} f_*(g \circ k) \stackrel{(1)}{=} \underbrace{f \circ g \circ k}_{(i) \Rightarrow 1_y} \stackrel{(i)}{=} k \end{aligned} \right\} g_* = (f_*)^{-1}
 \end{aligned}$$

TODO:

$$\begin{aligned}
 (ii) &\stackrel{!}{\Rightarrow} (i) \\
 g_* &= f_*^{-1}
 \end{aligned}$$

1.1.2.2.2. Split morphisms



$$x \xrightarrow{s} y \xrightarrow{r} x \wedge r \circ s = 1_x$$

$\Rightarrow s$ is split monomorphism

NOTE: that doesn't mean that $s \circ r = 1_y$.
If so, then they would be isomorph.

1.1.2.2.3. Exercises

1.2.i.

Goals:

$$(1) C/c \cong (c/(C^{\text{op}}))^{\text{op}}$$

$$(2) \exists \underbrace{c/C}_{C \text{ under } c} \Rightarrow \exists \underbrace{C/c}_{C \text{ over } c}$$

1.2.ii

$$(i) \text{ monic } f : x \rightarrow y \wedge g : y \rightarrow z \Rightarrow gf : x \rightarrow z$$

$$f : x \rightarrow y \Rightarrow h, k : w \rightarrow x, \underline{f \circ h = h \circ k \Rightarrow h = k} \quad !$$

$$(g \circ f) \circ h = (g \circ f) \circ k \Rightarrow h = k \quad !$$

$$\Leftrightarrow g \circ (f \circ h) = g \circ (f \circ k) \Rightarrow h = k$$

$$\Leftrightarrow g \circ (f \circ h) = g \circ (f \circ k) \Rightarrow h = k$$

g monic

$$\Leftrightarrow f \circ h = f \circ k \Rightarrow h = k$$

f monic

$$\Leftrightarrow h = k \Rightarrow k = k$$

(i') (dually)

(ii) monic $g \circ f, f$

$$f \circ h = f \circ k \Rightarrow h = k \quad !$$

$$\Leftrightarrow g \circ (f \circ h) = g \circ (f \circ k) \Rightarrow h = k$$

$$\Leftrightarrow (g \circ f) \circ h = (g \circ f) \circ k \Rightarrow h = k$$

$g \circ f$ monic

$$\Leftrightarrow h = k \Rightarrow h = k$$

(ii') (dually)

2. OSDEV

useful links & cool projects

[OSDev wiki](#) , [osdever.net](#) , [it's good to be king](#) , [9front \(Plan 9\)](#) , [mirageOS](#) , [Uxn](#) , [seL4](#) ,

2.1. KERNEL

2.1.1. Memory

Paging: Memory gets subdivided into fixed-size blocks called **pages**.

Segmentation: Memory gets subdivided into variable-sized segments based on logical units such as functions, arrays, or data structures

Virtual memory: Exposing a virtual continuous block of memory to user-level applications which gets is to the underlying separate physical parts

DMA: Direct Memory Access

2.1.2. Processes

2.1.2.1. Inter Process Communication (IPC)

Shared memory: Processes can use shared memory for extracting information as a record from another process as well as for delivering any specific information to other processes

Message passing: Processes communicate by sending and receiving messages to exchange data. Message Passing can be achieved through different methods like Sockets, Message Queues or Pipes

Gates:

IPC ports:

2.1.2.1.1. Multithreading/Synchronization

Synchronization: Maintaining the consistency of shared data even when multiple processes/threads are executed simultaneously.

Critical region: A region of memory shared by 2 or more processes

Lock: A variable whose value allows or denies the entrance to a critical region

Semaphore: A technique used to manage concurrent processes accessing a critical region by using a lock

Counting Semaphore (Semaphore): Allows access of a critical region to a group of processes

Binary Semaphore (Mutex): Allows access of a critical region to one process

Busy waiting: Continuously testing of a variable until some value appears

Spinlock: A lock which uses busy waiting

2.1.2.1.2. Multitasking

Round Robin: Processes are put into a circular queue and executed sequentially

Priority scheduling algorithm: Each process is allocated a priority and the process with the highest priority is executed first

2.1.3. Syscalls

2.1.4. CPU

Interrupts: Signals that prompt the processor to temporarily halt its current tasks to address an event

IRQ: Interrupt Request

ISR: Interrupt Service Routine

2.1.4.1. x86

GDT: Global Descriptor Table

IDT: Interrupt Descriptor Table

2.1.5. Filesystem

2.1.6. Hardware

2.1.6.1. Device drivers